Сбор данных и разметка: от формулировки задачи до крауда

В этой лекции поговорим про получение данных и **процесс разметки** — как **при помощи LLM**, так и **комбинированием подходов** или **классическим краудсорсом**, а также рассмотрим и **продвинутые подходы**.

Данные важны

Часто можно слышать, что данные — это 80% успеха при решении прикладной задачи. В этом есть большая доля правды — они определяют то, как модель поведёт себя в проде. К примеру, модель **GALACTICA** от Meta была свёрнута через три дня после запуска из-за её токсичных и фейковых ответов — одной из причин была слабая подготовка обучающих данных.

Сбор данных — это ещё и дорого и долго. По оценке Semianalysis, OpenAI потратили десятки миллионов долларов на сбор разметки для GPT. А то, как много времени занимает поиск разметчиков, согласование инструкций, валидация и правки, вы наверняка могли узнать на практике.

В общем, получение и разметка данных — сложный, но важный процесс, который мы и обсудим!

LLM-разметка

Методы работы с данными эволюционируют очень быстро, и в последние годы разметка всё чаще автоматизируется с помощью LLM (или VLM):

- ChatGPT > разметчики MTurk на текстовой разметке;
- RLAIF достигает качества RLHF без ручных меток;
- автоматизация разметки в поиске Яндекса: 105% качества относительно людей, −60 % затрат;

- всё больше моделей обучаются только на синтетике и экономят на разметке.

Преимущества такого подхода:

- срок вывода модели в прод падает в разы;
- стоимость разметки падает;
- высокая гибкость: быстрое изменение инструкций;
- улучшение качества, гибридные пайплайны.

Посмотрим на конкретные данные:

	gpt-4 turbo	Асессоры
Цена микротаски (базовой единицы задания)	8 ₽	XX.XX ₽
Среднее число разметок на 1 пример	~2.44	~4
Средняя цена 1 разметки	~22 ₽	~XXX ₽
Среднее время разметки 200 примеров	0.5 ч	2–∞ ч
Дополнительные расходы	~~pretrain & alignment~~	доп. инструкции
Время обучения	~1 с	>1 д
Сложность изменения инструкции	~мало	~много

LLM-разметка на примере кейса

Разберём процесс такой разметки на примере несложной задачи: предположим, от бизнеса пришло требование: определить, нужно ли отвечать на запрос пользователя в поисковой системе/приложении отдельной генеративной моделью для математических примеров — то есть, сделать классификацию школьных математических задач.

На первый взгляд это просто, но при анализе негативов появляется много нюансов:

Инстракт	**Вердикт**	**Причина**
∫_(-π/3)^(π/6)(3/sin^3(x))dx	👎🏽	не школьное
Два ребра прямоугольного параллелепипеда равны 1 и 2 а объем равен	👎🏽	неполное условие

Инстракт	Вердикт	Причина
5 джоули в электрон-вольты	👎🏾	физика
5 евро в рублях	👎🏾	нужна актуальная информация

Первый шаг разметки — **подбор первичного промпта**:

- берём имеющуюся инструкцию для разметки;
  - если её нет, начинаем с нескольких предложений, где просим модель разметить данные;
  - можно и делегировать составление промпта на LLM (например, с помощью system prompt generator).

Первый шаг: подбор первичного промпта

Дальше мы сталкиваемся с не совсем ожидаемыми ответами, так как короткий промпт не передаёт точное видение разметки:

- много False Positive (все с упоминанием математики);
- модель не понимает, что именно относить к школьным математическим задачам.

Поэтому переходим ко второму шагу — **тесту и первичному тюнингу**:

- разбиваем на подзадачи попроще — добавить многоклассовую классификацию тематик школьных задачек;
- просим GPT со специальным систем промтом поправить инстракт.

Второй шаг: тест и первичный тюнинг

Третьим этапом **мы добавляем автометрики** и **больше примеров** (около 100) — мы хотим понять, как часто модель ошибается и как мы можем на это повлиять изменением промпта. Собственно автометрики — это всё, что мы можем посчитать автоматически, например:

- качество по оценкам квалифицированных и обычных разметчиков: f1, precision, recall;
- погрешности (расхождения между разными запусками на одном примере);
- % ответов не в нужном формате;
- наличие байесов (e.g. в сторону самого первого или самого длинного ответа);
- метрики на подзадачах, разбивки на простые/сложные примеры, ...

Шаг №3: замер качества и автометрики

И после всего этого итеративно улучшаемся:

Последний шаг — итеративное улучшение

Как улучшать LLM-разметку: промпт-инженерные хаки & трюки

Улучшаться можно по-разному. Может быть полезен промптинг **Chain of Thought**: просим модель ответить не сразу, а предварительно выписав рассуждения и выдать вердикт в виде цифры только после них. Дополнительный бонус: мы можем смотреть на рассуждения модели и понимать, что идёт не так. Из минусов — заметно растёт стоимость генерации.

Удобно просить **вывод в виде JSON**:

- структура `{"reasoning": ..., "verdict": ...}`;
- можно попросить и дополнительные аргументы для удобного парсинга.

Можно попробовать **разные модели**:

Класс	Примеры (≈$/1k ток)	Сильные стороны	Ограничения
Proprietary top-tier	GPT-4.1, Claude 3 Opus (>$10)	Лучший reasoning, низкий error-rate (особенно для сложных edge-кейсов).	Дорого, нельзя хранить PII, скорость через API.
Open-source >30 B	Qwen3, DeepSeek-R1, DeepSeek-V3	Почти GPT-4-level	Требует > 1×A100, latency ↑.
Mid-range 7–13 B	Mistral-7B, Zephyr-β, Gemma-7B-IT	Дешевле ($0.05–0.1), хватает на простую разметку, легко запускать несколько для голосования.	Хуже на длинных контекстах, больше false-positive.

Если мы хотим подсветить спорные моменты, поможет техника **Few-Shot** — добавим в промпт несколько примеров. Это полезно, когда мы хотим подчеркнуть особенности инструкции или ищем ошибки. Важно учитывать несколько факторов:

- балансировать positives/negatives;
- избегать наивные паттерны (pos/neg/pos/neg).

В длинном контексте разобраться тяжело. Можно **перебрать формат входа** (или даже всего промпта) — к примеру, может оказаться, что модели тяжело видеть формат `"doc1 fact1 doc2 fact2"`. Иногда бывает легко повысить качество за счёт **ризонинг моделей** (начинать стоит с open-source решений).

Нужно уметь **бороться с шумом**:

- для **стабильности** можно выставить `temperature = 0`, `top_p = 0.0001`;
- **self-consistency**: можно качать несколько сэмплов (`n=3`) и агрегировать вердикты (major vote), что может дать рост качества (например, когда одна модель даёт лучший Precision, а вторая — лучший Recall).

А также с **байесами и инжектами** — из-за особенностей обучения у LLM могут быть предпочтения по длине или позиции, и в промышленных условиях это может сыграть злую шутку. Что с этим можно сделать:

- **байес на длину** — фикс инструкции;
- **байес позиции** — два способа:
  - качаем пару (A,B) и пару (B,A), смотрим на совпадение вердиктов;
  - если не совпал, делаем ещё одну генерацию, где просим ту же модель (или другую модель-судью) прочитать размышления и сделать свой вывод.

Ещё вы наверняка могли слышать про подкуп моделей словами (e.g. пообещать денег), но на самом деле сейчас это уже не помогает.

Не забывайте, что это **всё ещё задача классического ML**, поэтому важно:

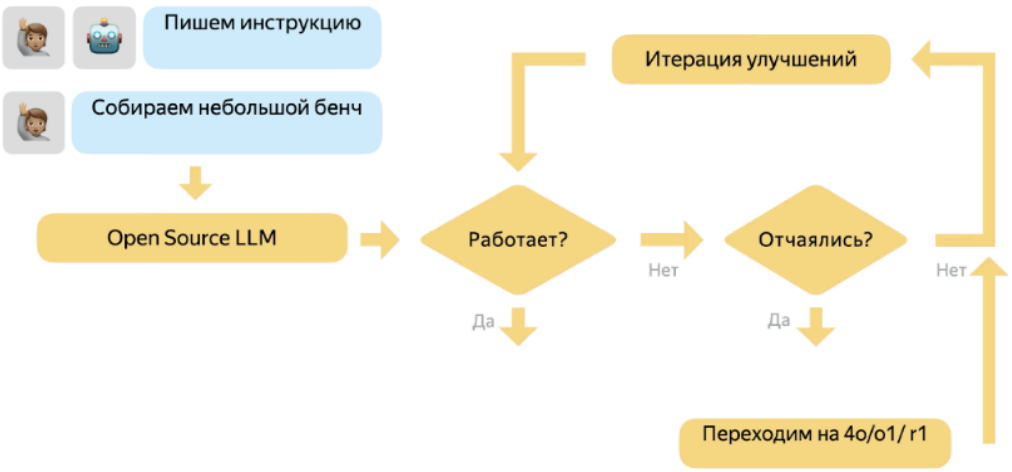
- оценивать шум;
- хорошо шаффлить данные и не забывать про их распределения;
- бояться переобучения и откладывать пулы.

И ещё немного советов:

Приём	Короткое объяснение
Жёсткая output-schema (JSON Schema, Pydantic)	Валидный формат → меньше ошибок парсинга (даже при prompt injection)
Температура 0, top_p ≈ 0	Детерминированность → воспроизводимые ответы
Few-shot / примеры	Калибровка того, «как решать» + помогает фиксить edge-кейсы
Добавление контекста домена, обогащение входных данных	Термины и определения прямо в промте → меньше галлюцинаций
Совпадение train/val распределений с требованиями (prod)	Задаём выборки без domain drift → корректные метрики
Версионирование промптов и сетапа инференса (например, в Git)	Воспроизводимость, а также ревью и отслеживание

Приём	Короткое объяснение
Голосование и перекрытие: одной или нескольких LLM	Ансамбль моделей → уменьшение случайных ошибок, увеличение качества, метрика согласованности
Shuffle, hold-out, классические ML-проверки	Контроль переобучения и утечек

Если у вас ограниченный бюджет, важно посчитать цену токенов входа и выхода; стоит прогнать тестово прогнать n примеров и посчитать, сколько это будет стоить. Полезно прогнать и через несколько моделей и/или несколько раз.



Итоговый алгоритм запуска разметки

Итог

- LLM можно эффективно использовать для разметки данных через продуманный промпт;
- качество зависит от формулировки, примеров, формата, модели;
- оценка важна с самого начала: автометрики + ручная проверка.

Практические советы:

- делайте первые версии датасетов с помощью самой же LLM;
- тестируйте разные промпты и версии модели;

- валидация и разбор ошибок обязательны.

Ручная разметка и гибкие пайплайны

Даже с идеальными промптами и лучшими моделями вы в какой-то момент упрётесь в потолок качества. На простых задачах как в нашем примере достичь хорошего качества можно быстро. Но если в вашем кейсе важен контекст, есть неоднозначность или нет чёткого согласия у людей, выбить качество только за счёт промпта будет тяжело.

Сигналы, что нужна ручная разметка:

- **Потолок качества**: Δ метрики < X pp за 3-5 итерации промпта.
- **Особенности контекста и задачи**: специфический либо широкий контекст, случаи, когда нужно интерпретировать поведение пользователя в контексте.
- **Высокая неоднозначность**: согласованность между людьми <80%.
- **Критически важные метрики, разметка прода**: e.g. оценка, насколько модели «врут» в продукте.

Комбинированный подход — золотая середина

Итак, мы столкнулись с тем, что задача не покрывается LLM полностью. Во множестве статей мы видим комбинированный подход между LLM и людьми:

- GPT-4 + крауд растит качество в сравнении с только краудом: 83.6 → 87.5;
- В статье смешивание синтетики и ручной разметки экономит при том же F1: $1600 → $200.

Когда доверяем LLM:

- однозначные случаи, высокая согласованность на примерах;
- несложные задачи;
- уверенность модели / автометрики проходят порог.

Когда отдаём квалифицированным разметчикам:

- edge-кейсы, неоднозначность, метки не не согласованы;
- низкая уверенность модели;
- требуется дополнительный контекст.

Так мы сможем растить качество и снижать бюджет.

По сути выбор между LLM и человеческой разметкой сводится к разделению примеров на простые и сложные:

- гейт по уверенности;
- Active Learning;
- постоянный мониторинг качества.

Вот бейзлайн для определения сложности на основе уверенности:

- Считаем уверенность: `conf = max softmax` или `1 - entropy`.
- Два простых порога:
  - 0.8: легко ⇒ размечает LLM;
  - 0.5–0.8: сомнительно ⇒ LLM + 1 эксперт;
  - < 0.5: сложно ⇒ 2–3 эксперта.
- Итеративно доучиваем модель на «сложных» примерах.

Классический крауд

Могут быть ситуации, когда LLM совсем плохо справляется с задачей / даёт нестабильное качество — тогда можно вернуться к классическому крауду. В этом случае очень важно сделать разметку максимально системной и управляемой. Посмотрим на общий пайплайн разметки.

1. Требования бизнеса. Начинаем с формулировки задачи и требований к данным, формируем дизайн-документ:

- какие данные/разметка;
- зачем они нужны;
- какой объём;

- требуемое качество и оптимизируемая метрика.

2. Разработка инструкции. Здесь процесс похож на то, о чём мы говорили в начале, только вместо LLM — асессоры, и мы итеративно улучшаем инструкцию:

- **самостоятельная проба** (10-30 примеров): ловим все edge-кейсы, заносим в инструкцию;
- **разметка людьми внутри команды / квалифицированными разметчиками**: размечаем с перекрытием, оцениваем согласованность и фиксим расхождения;
- **инструкция v0**: примеры «хорошо/плохо/граница», чёткий формат ответа;
- **итерации улучшения**: получаем фидбек от разметчиков и их данных, учитываем их частые ошибки.

Важно достичь согласованности хотя бы на небольшом количестве примеров (10-30) с командой/аналитиками/заказчиками; затем можно идти к человеку вне контекста уже с инструкцией — понять, насколько она ясна. И уже после этого мы переходим к асессорам и разметке на крауд-платформе.

3. Data flow + крауд + UI. На этом этапе решаем организационные вопросы:

- где берём сырые данные, какое распределение;
- этапы обработки (e.g. нужно ли обезличивание), фильтрации, feature extraction, метаданные;
- как должен выглядеть интерфейс задания;
- кто будет размечать: квалификация разметчиков, постоянная ли рабочая группа.

Сервисов для крауда достаточно много, вот одни из самых популярных:

Крауд платформа	Для чего
Яндекс.Задания	Текст/изображения, СНГ
Toloka	Текст/изображения, мир
Amazon Mechanical Turk	Массовый крауд, мир
Open source сервис	

Label Studio	Open source, любые
Doccano	Open source, текст, sequence lab
Оркестрация пайплайнов	
n8n	No-code пайплайны
Airflow	Интеграция с данными / ETL

4. Процесс разметки: контроль качества и процесса. Наконец переходим к самой разметке.

Асессора нужно обучить (5-10 примеров с пояснениями), а затем проверить, как хорошо он умеет решать задание (автосчёт баллов с порогом). Для расчёта порога можно прогнать через экзаменов N асессоров и оттолкнуться от того, какое качество необходимо и какой процент асессоров подходит.

Важно контролировать качество (на случай, если асессор начинает читерить):

- ханипоты — скрытые проверочные задания;
- статистика по каждому ассесору: точность, время, резкие скачки;
- агрегация и перекрытие: дать задачу 2-3 разным людям и посчитать согласие между ними; если не сошлись — передать на редактора или старшего ассессора

5. Итерации улучшения. Лучше короче, но больше.

Продвинутые практики

Напоследок — ещё несколько продвинутых практик, которые помогут в комбинированном крауде.

Приём	Идея	Плюс
1. Знание о качестве ассессоров	weight = trust(worker)	Сильные метки влияют ↗, шум уходит ↘
2. Active Learning	Отдаём людям только Low-conf	Экономим X % бюджета, качество растёт
3. Дашборд ошибок	LLM vs ассессоры: где расхождения?	Видно, что править в инструкции

Выводы

💡 Работа над данными — это не этап до старта модели. Это постоянная итеративная часть жизненного цикла продакшн ML-системы.

Эффективный инструмент — автоматическая разметка через LLM:

- гибрид с экспертами;
- active learning и переобучение по «сложным» примерам.

Это сокращает затраты на разметку в несколько раз, ускоряет ввод в эксплуатацию и существенно снижает риски деградации и багов. Поэтому:

1. Стройте пайплайн данных так же серьёзно, как и код: версия данных, тесты, метрики, мониторинг.
2. Используйте LLM там, где это уже даёт приемлемое качество, а экспертов — для более сложных кейсов.
3. Замкните петлю: поступают новые данные → метки → дообучение → контроль качества → снова в продакшн.